

REPORT ABOUT INDEX IN ORACLE DBMS

Prepared by:

B2205896

B2205976

B2203438

B2205983

Nguyễn Minh Nhựt

Trần Hữu Giang

Huỳnh Tấn Đạt

Đỗ Đạt Hoa

Instructor: Trần Công Ân

Course: Oracle Database Management System

1. What is an index?

Definition

An **index** is an optional structure, associated with a table or table cluster, that can sometimes speed up data access.

Indexes are schema objects that are logically and physically independent of the data in the objects with which they are associated. Thus, you can drop or create an index without physically affecting the indexed table.

Benefits of using indexes in Oracle:

1. **Increased query speed:** Helps retrieve data faster by reducing the number of rows to scan.
2. **Reduced processing time with large datasets:** Especially useful for searching in large tables.
3. **Improved join performance:** Speeds up joins between multiple tables in queries.
4. **Faster range queries:** Efficient for range queries like BETWEEN, >, <.
5. **Reduced resource usage (I/O and CPU):** Indexed queries avoid reading unnecessary data.

How to work with index in Oracle:

- To **create** a new index for a table, you use the **CREATE INDEX** statement as follows:

```
CREATE INDEX index_name ON table_name(column1[, column2,...]);
```

Ex:

```
CREATE INDEX idx_employee_name ON employees(last_name);
```

```
CREATE INDEX idx_emp_last_dept
```

```
ON employees(last_name, department_d);
```

Or you can also create index for each index type:

```
CREATE BITMAP INDEX idx_emp_gender ON employees(gender);
```

```
CREATE INDEX idx_emp_lower_last_name
```

```
ON employees(LOWER(last_name));
```

- To **view** all indexes of a table, we query from the all_indexes view:

```
SELECT index_name, index_type, visibility, status
FROM all_indexes
WHERE table_name = 'MEMBERS';
```

- To **remove** an index, we use the **DROP INDEX** statement:

```
DROP INDEX index_name;
```

Ex: **DROP INDEX** idx_employee_name;

- To use an index:

```
SELECT * FROM employees WHERE last_name = 'Pain';
```

2. What types of index does Oracle support?

You can use multiple types of indexes depending on the nature of the table and data:

- B-Tree indexes are the default and most common.
- Bit-mapped indexes are used for data with low cardinality.
- Function-based indexes exist on functions on SQL statements.
- An application domain index is a customized index specific to an application.

Indexes are key to fast data access but come at a cost. The index must be updated every time data is inserted, updated, or deleted; this is a performance hit. Indexes also consume disk space and are commonly stored in tablespaces separate from the corresponding data tablespaces.

B-Tree Indexes

B-trees, short for *balanced trees*, are the most common type of database index. A **B-tree index** is an ordered list of values divided into ranges. By associating a key with a row or range of rows, B-trees provide excellent retrieval performance for a wide range of queries, including exact match and range searches.

Usage

- Suitable for columns with **high cardinality** (many distinct values), like an ID or product code.
- Best for queries involving equality (=) or range conditions (BETWEEN, >, <).
- Effective in systems where frequent data updates or modifications occur.

Bitmap Indexes

In a **bitmap index**, the database stores a bitmap for each index key. In a conventional B-tree index, one index entry points to a single row. In a bitmap index, each index key stores pointers to multiple rows.

Each bit in the bitmap represents a row in the table. If the bit is set to 1, it means the corresponding row contains the index key value. Conversely, if the bit is set to 0, the row does not contain the key value.

Usage

- Suitable for columns with **low cardinality** (few distinct values), like boolean flags, status fields, or gender.
- Optimized for **read-heavy** systems with fewer data modifications, like reporting systems or data warehouses.
- Bitmap Indexes excel in complex queries with multiple AND and OR conditions, as they can combine bits efficiently for filtering.

Function-Based Indexes

A **function-based index** computes the value of a function or expression involving one or more columns and stores it in an index. A function-based index can be either a B-tree or a bitmap index.

The indexed function can be an arithmetic expression or an expression that contains a SQL function, user-defined PL/SQL function, package function, or C callout. For example, a function could add the values in two columns.

This type of index is particularly useful when you frequently query columns using functions or expressions (e.g., UPPER(), LOWER(), SUBSTR(), or mathematical operations).

Usage

- Columns that are frequently **transformed** or **calculated** in queries (e.g., converting uppercase to lowercase, or date columns used in mathematical operations).

- Complex queries involving **SQL functions** (e.g., LOWER(), UPPER()), arithmetic expressions, or user-defined functions.
- Useful for indexing **complex expressions** rather than just raw column data.

Application Domain Indexes

An **application domain index** is a customized index specific to an application.

Extensive indexing can:

- Accommodate indexes on customized, complex data types such as documents, spatial data, images, and video clips (see "[Unstructured Data](#)").
- Make use of specialized indexing techniques.

You can encapsulate application-specific index management routines as an index type schema object, and then define a domain index on table columns or attributes of an object type. Extensible indexing can efficiently process application-specific operators.

Usage

- **Unstructured or complex data** such as text, spatial data (geographic coordinates), images, and videos.
- Suitable for **specialized searches** that standard indexes cannot handle (e.g., text search, spatial search, image data analysis).
- Used in **specialized applications** that require domain-specific processing.

Partitioned Indexes

Similar to partitioned tables, partitioned indexes improve manageability, availability, performance, and scalability. They can be partitioned independently (global indexes) or automatically linked to the table partitioning method (local indexes). Oracle Database supports both range and hash-partitioned global indexes. In a range-partitioned global index, each index partition contains values defined by a

partition bound. In a hash-partitioned global index, each partition contains values determined by the Oracle Database hash function.

3. Demonstrate the effect of index in querying data in Oracle (Hint: using the execution plan).

An **execution plan** describes a recommended method of execution for a SQL statement. The plan shows the combination of the steps Oracle Database uses to execute a SQL statement. Each step either retrieves rows of data physically from the database or prepares them for the user issuing the statement.

An execution plan displays the cost of the entire plan, indicated on line 0, and each separate operation. The cost is an internal unit that the execution plan only displays to allow for plan comparisons. Thus, you cannot tune or change the cost value.

To demonstrate how the presence of indexes can improve the performance of SQL queries in Oracle databases by comparing execution plans before and after index creation.

Load the employees table with a sufficient amount of sample data (about 100,000 rows) to effectively demonstrate the impact of indexing.



```
Command Prompt - sqlplus
SQL> desc employees;
  Name                                Null?    Type
  ----                                -
EMP_NO                                NOT NULL NUMBER
FIRST_NAME                           VARCHAR2(50)
LAST_NAME                            VARCHAR2(50)
EMAIL                                VARCHAR2(100)
PHONE                                VARCHAR2(20)
SALARY                               NUMBER(10,2)

SQL> select count(*) from employees;

  COUNT(*)
  -----
      85692

SQL>
```

Run a query that searches for non-primary key columns phone to establish a baseline for performance.

```

C:\> Command Prompt - sqlplus
SQL> EXPLAIN PLAN FOR
2 select first_name, last_name, phone from employees where phone = '0923338088';
Explained.

SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 1445457117

-----
| Id | Operation          | Name          | Rows | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT   |               |      8 |    504 |    202  (1)| 00:00:03 |
|*  1 |  TABLE ACCESS FULL| EMPLOYEES     |      8 |    504 |    202  (1)| 00:00:03 |
-----

Predicate Information (identified by operation id):
-----
PLAN_TABLE_OUTPUT
-----
      1 - filter("PHONE"='0923338088')

Note
-----
   - dynamic sampling used for this statement (level=2)

17 rows selected.

SQL>

```

Review the execution plan returned by the **EXPLAIN PLAN** statement. Look for indicators of a **Full Table Scan**, which indicates that the database is scanning the entire table to find matching rows.

Create an index on the phone column to optimize future queries. After creating the index, re-run the same query and use **EXPLAIN PLAN** again to analyze the new execution plan.

```

C:\> Command Prompt - sqlplus
SQL> CREATE INDEX idx_employees_phone ON employees(phone);
Index created.

SQL> EXPLAIN PLAN FOR
  2 select first_name, last_name, phone from employees where phone = '0923338088';
Explained.

SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 4076964109

-----
| Id | Operation                      | Name                | Rows | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT                |                     |      1 |    27 |      2 (0)| 00:00:01 |
|  1 | TABLE ACCESS BY INDEX ROWID    | EMPLOYEES           |      1 |    27 |      2 (0)| 00:00:01 |
|*  2 | INDEX RANGE SCAN                | IDX_EMPLOYEES_PHONE |      1 |      |      1 (0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
PLAN_TABLE_OUTPUT
-----

      2 - access("PHONE"='0923338088')

14 rows selected.

SQL>

```

Focus on the number of rows examined. Whether the execution plan shows an **Index Range Scan** instead of a **Full Table Scan**. The overall cost of the query execution with the index on the phone column is much better than no index.

To further demonstrate the effect of indexes in Oracle, you can follow this extended plan. Create an index on the **first_name** column, which has many duplicate values.

```

C:\> Command Prompt - sqlplus
SQL> SELECT COUNT(*) AS duplicate_name_count
  2 FROM (
  3     SELECT first_name
  4     FROM employees
  5     GROUP BY first_name
  6     HAVING COUNT(*) > 1
  7 );

DUPLICATE_NAME_COUNT
-----
                  690

SQL> drop index idx_employees_phone;
Index dropped.

```

Execute a query that searches for a specific **first_name** to see the effect of the index.


```

Command Prompt - sqlplus
SQL> CREATE INDEX idx_employees_fname ON employees(first_name);
Index created.

SQL> EXPLAIN PLAN FOR
  2 select first_name, last_name, phone from employees where first_name = 'Jacqueline';
Explained.

SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 1370298926

-----
| Id | Operation              | Name              | Rows  | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT        |                   |    124 |   3348 |   99  (0)| 00:00:02 |
|  1 | TABLE ACCESS BY INDEX ROWID | EMPLOYEES         |    124 |   3348 |   99  (0)| 00:00:02 |
|*  2 | INDEX RANGE SCAN        | IDX_EMPLOYEES_FNAME |    124 |         |    1  (0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
PLAN_TABLE_OUTPUT
-----

      2 - access("FIRST_NAME"='Jacqueline')

14 rows selected.

SQL>

```

Since **first_name** contains many duplicate values, the cost of searching may not decrease significantly because the database still has to scan multiple rows for matches. So, the cost doesn't decrease though it has an index.

Drop all indexes and go to the next example:

```

Command Prompt - sqlplus
SQL> EXPLAIN PLAN FOR
  2 SELECT first_name, last_name, phone FROM employees WHERE first_name = 'Jacqueline' AND phone LIKE '%8685';
Explained.

SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 1445457117

-----
| Id | Operation          | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT   |           |      6 |   162 |   202  (1)| 00:00:03 |
|*  1 | TABLE ACCESS FULL| EMPLOYEES |      6 |   162 |   202  (1)| 00:00:03 |
-----

Predicate Information (identified by operation id):
-----
PLAN_TABLE_OUTPUT
-----

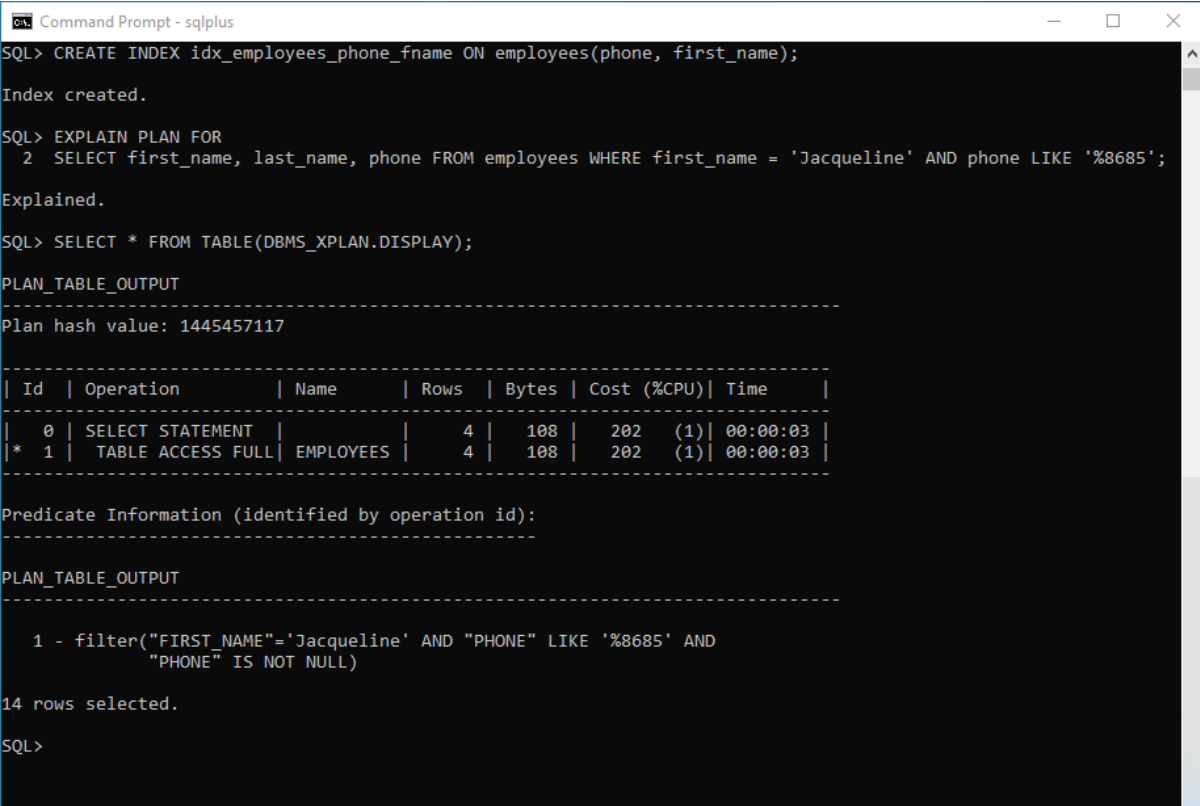
      1 - filter("FIRST_NAME"='Jacqueline' AND "PHONE" LIKE '%8685' AND
              "PHONE" IS NOT NULL)

14 rows selected.

SQL>

```

Create a composite index that includes both **phone** and **first_name**. Execute a query that searches with both **first_name** and **phone**.



```
SQL> CREATE INDEX idx_employees_phone_fname ON employees(phone, first_name);
Index created.

SQL> EXPLAIN PLAN FOR
  2 SELECT first_name, last_name, phone FROM employees WHERE first_name = 'Jacqueline' AND phone LIKE '%8685';
Explained.

SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 1445457117

-----
| Id | Operation          | Name                | Rows  | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT    |                     |      4 |    108 |    202 (1)| 00:00:03 |
|*  1 |  TABLE ACCESS FULL | EMPLOYEES           |      4 |    108 |    202 (1)| 00:00:03 |
-----

Predicate Information (identified by operation id):
-----
PLAN_TABLE_OUTPUT
-----
   1 - filter("FIRST_NAME"='Jacqueline' AND "PHONE" LIKE '%8685' AND
            "PHONE" IS NOT NULL)

14 rows selected.

SQL>
```

The effectiveness of the index has not been maximized for this query, especially due to the usage of **LIKE '%8685'**. This type of pattern matching can lead to full table scans, as the leading wildcard (%) prevents the optimizer from utilizing the index efficiently.

Create another composite index that includes both **first_name** and **phone**.

```

Command Prompt - sqlplus

SQL> CREATE INDEX idx_employees_fname_phone ON employees(first_name, phone);
Index created.

SQL> EXPLAIN PLAN FOR
  2 SELECT first_name, last_name, phone FROM employees WHERE first_name = 'Jacqueline' AND phone LIKE '%8685';

Explained.

SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 3537337632

-----
| Id | Operation                      | Name                      | Rows | Bytes | Cost (%CPU)| Time |
-----+-----+-----+-----+-----+-----+-----+
|  0 | SELECT STATEMENT                |                           |      4 |  108 |       7 (0)| 00:00:01 |
|  1 | TABLE ACCESS BY INDEX ROWID    | EMPLOYEES                 |      4 |  108 |       7 (0)| 00:00:01 |
|*  2 | INDEX RANGE SCAN                | IDX_EMPLOYEES_FNAME_PHONE |      4 |      |       2 (0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
PLAN_TABLE_OUTPUT
-----
-----
      2 - access("FIRST_NAME"='Jacqueline')
         filter("PHONE" LIKE '%8685' AND "PHONE" IS NOT NULL)

15 rows selected.

```

The execution plan for this query likely showed different statistics compared to the first query. The new order of columns in the composite index allowed the database to efficiently filter by **first_name** first and then check the **phone** values. If **first_name** significantly reduces the number of rows to evaluate, it enhances performance, even with the **LIKE** condition on **phone**.

Through these experiments, it has effectively illustrated the impact of indexing on query performance in Oracle. The results reinforce the importance of thoughtful index design and the significance of query structure in achieving optimal performance.

4. References

- SQL Tuning Guide: [Oracle SQL Tuning Guide](#)
- Database Tuning: [Database Tuning Book](#)
- Indexes and Index-Organized Tables: [Oracle Documentation](#)
- SQL Indexing and Tuning: [Use the Index, Luke](#)
- Oracle Indexes Reduce The Performance: [Bytescout Blog](#)
- Indexing LIKE Filters: [Use the Index, Luke](#)